

Running IT for a Team You Never See

What changes when your people are in six states and your users are everywhere

BY RASHAD MORGAN

Most advice about remote work is about remote work. It assumes a team that used to sit together and now does not, and it is mostly about preserving what was lost.

That is not the situation I am describing. The situation I am describing is a support organization that was never in one place, serving a user population that spans time zones, where the nearest colleague to any given technician might be six hundred miles away. Nothing was lost, because nothing was ever co-located. The question is not how to preserve the office. It is how to run a service when proximity was never available to lean on.

PROXIMITY WAS HIDING THINGS

The first thing you notice is that distance does not create problems so much as expose them.

When a team sits together, a great deal of coordination happens invisibly. Someone overhears a call and offers context. A technician turns around and asks the person behind them. A manager sees somebody struggling and walks over. None of it is written down and none of it appears in any process document, because nobody had to design it.

Remove proximity and that layer disappears. What is left is whatever you actually built.

So the failures you see in a distributed team are rarely failures of distance. They are gaps in documentation, in escalation paths, in role clarity, that were being patched by physical adjacency. The distance did not break anything. It stopped concealing what was already broken.

This is worth knowing because it changes what you fix. The instinct is to add communication: more meetings, more check-ins, more presence. Usually the right answer is to make explicit the thing that used to happen implicitly.

WRITE THINGS DOWN, AND MEAN IT

Documentation in a co-located team is a nice-to-have that everyone agrees is important and nobody does. In a distributed team it is the substrate.

The distinction that matters is between documentation that describes a system and documentation that lets someone act. Most internal documentation is the first kind. It tells you how something is configured. What a technician at 6am in another time zone needs is the second kind: what to do, in what order, and what to do when the expected thing does not happen.

A few things that make the difference:

Write for the person who has the problem, not the person who built the system. The article should start from the symptom, not the architecture.

Include the failure paths. Any runbook can describe the happy path. What makes one useful at 2am is the section explaining what it looks like when step four does not work.

Date it and own it. Documentation with no owner and no date rots invisibly, and rotted documentation is worse than none, because someone will follow it.

Accept that it is a real cost. Writing this well takes time that is not spent closing tickets. A team that never gets that time will never have it, and will keep paying for the absence in escalations that should not have been escalations.

AVAILABILITY IS A DESIGN DECISION, NOT A SCHEDULE

When your users span time zones, somebody is always starting their day while somebody else is finishing.

The naive response is to extend hours until the coverage gap closes, which converts a scheduling problem into a staffing problem and a retention problem. The better response is to decide, deliberately, what actually needs live coverage and what does not.

Most support volume is not time-critical. A request that arrives at 7am in one region and gets handled at 9am in another is fine, as long as the user knows that is what will happen. The unacceptable outcome is not delay, it is uncertainty. People plan around a known response window. They cannot plan around silence.

That leads to a small number of real decisions:

- Which categories genuinely need someone awake, and which need only acknowledgment
- Where the boundary sits between regions, and who holds the handoff
- What "urgent" means, written down, so it is not decided by whoever shouts
- How an incident reaches a human outside business hours, and who that human is

None of this is complicated. It is just easier to avoid deciding and let the schedule absorb the problem, which works until the people absorbing it leave.

HANDOFFS ARE WHERE THE SERVICE ACTUALLY LIVES

In a distributed team, work crosses people more often. Every crossing is a chance to lose context.

The cost is invisible in the metrics. A ticket handed between three technicians across two regions can still close within SLA. The user experienced three separate conversations, re-explained the problem twice, and formed the impression that nobody knew what was going on. Your dashboard says the service performed well.

What helps:

Write the ticket for the next person. Not a log of what you did, a statement of where things stand: what has been ruled out, what is still open, what the user has already been told.

Hand off to a person, not a queue. A named handoff creates accountability. A queue creates a period where nobody owns it.

Measure reassignments. It is a better proxy for service quality than resolution time, and almost nobody tracks it.

YOU CANNOT SEE WHO IS STRUGGLING

This is the part that is genuinely harder at a distance, and I do not think there is a clean solution.

In an office, you notice. Someone is quiet, or short with people, or working late in a way that is not normal for them. You do not need a process to catch it. At a distance, a person can be unhappy for months and the only signal is a resignation.

Partial mitigations, none of them complete:

Regular one to one time that is not a status update. If the only recurring conversation is about tickets, the only thing you will hear about is tickets.

Watch the shape of someone's work, not just the volume. Output often holds steady right up until it does not. Changes in what someone picks up, or stops picking up, or how they write, tend to precede changes in how much they do.

Make it normal to say the job is hard. If nobody on the team has ever said it out loud, the first person to do so is taking a risk, and most people will not.

Get in a room occasionally if you can. It is expensive and it is not always possible. It also resets something that video does not, and the effect lasts for months.

I want to be honest that this is the area where I am least confident. A distributed manager is working with less information about their people than a co-located one, and no amount of process fully closes that gap.

STANDARDS TRAVEL, CULTURE DOES NOT

A distributed team will not have one culture by default. It will have as many micro-cultures as it has locations, shaped by who was there first and what the local business does.

Some of that is fine and worth leaving alone. A technician embedded at a manufacturing site will work differently from one supporting a corporate office, and forcing uniformity on that is a waste of effort that also tells them their context does not matter.

What has to be uniform is narrower than most managers assume:

- What good service looks like and how it is measured
- How work gets prioritized when everything is urgent
- What gets escalated and to whom
- What is not negotiable, meaning security, access, and data handling

Beyond that, let people work the way that fits where they are. The energy spent enforcing uniformity on things that do not matter is energy not spent on the things that do.

WHAT THIS ACTUALLY BUYS YOU

The reason to get good at this is not that remote is pleasant. It is that a function built to work at a distance is a function that can absorb change.

A team that depends on proximity breaks when it grows, when it acquires, when someone moves. A team that never had proximity has already solved for it. Adding a new site is a configuration change rather than a reorganization. Absorbing another company's support staff is possible because your operating model never assumed everyone could see each other.

The discipline is the same discipline that makes any operations function scale: make the implicit explicit, decide things deliberately, and write them down. Distance just removes the option of skipping it.

Rashad Morgan leads shared services IT for a multi-brand industrial distribution group, with a team across six states supporting a global user base. His part of the work is identity, service management, and end user support.

[More writing](#)